SECUREFLOW

Automated DevSecOps Platform for Continuous Vulnerability Detection

PRODUCT REQUIREMENTS DOCUMENT

Document detail	Specification
Version	2.0 — Python Backend Edition
Status	Updated product and engineering baseline
Project type	B.Tech CSE final-year project / SaaS MVP
Domain	DevSecOps, application security, cloud infrastructure
Target schedule	Six-month implementation
Frontend	React + TypeScript + Tailwind CSS
Backend API	Python + FastAPI
Queue and workers	Celery + Redis; isolated Python scanner worker
Database	MongoDB + PyMongo
Security tools	Trivy + Gitleaks

Product promise: Connect a GitHub repository, scan supported security risks when code changes, and give developers a prioritized report before changes are merged.

This revision preserves the original product scope and security requirements while replacing the Node.js/Express API and BullMQ queue with a Python/FastAPI backend and Celery/Redis task processing.

Prepared: November 2026 | Supersedes PRD v1.0 for implementation technology

Document control and contents

Version	Change
1.0	Initial complete product requirements and engineering baseline.
2.0	Backend changed to Python/FastAPI; BullMQ replaced with Celery; Python project structure, task execution and API conventions updated.

• 1. Executive summary and product vision

• 2. Problem statement and opportunity

• 3. Goals, non-goals and success metrics

• 4. Users, roles and permissions

• 5. Product scope and prioritization

• 6. User journeys and system workflow

• 7. Functional requirements

• 8. Frontend and UX requirements

• 9. Architecture and service boundaries

• 10. Scanning engine and finding lifecycle

• 11. Data model and retention

• 12. API requirements

• 13. Security, privacy and threat controls

• 14. Non-functional requirements

• 15. Testing and acceptance criteria

• 16. Deployment and operations

• 17. Six-month implementation roadmap

• 18. Risks, assumptions and dependencies

• 19. Deliverables and demo plan

• 20. Glossary and technical references

1. Executive summary

SecureFlow is a web-based DevSecOps platform that integrates with GitHub repositories and runs selected security checks during software development. A user creates a workspace, installs the SecureFlow GitHub App on selected repositories, and enables monitoring. A supported pull-request or push event, or a manual request, starts an asynchronous scan.

The Python FastAPI backend validates incoming requests, authorizes users, records scan metadata and dispatches background work through Celery using Redis as the broker. A restricted scanner worker obtains a source snapshot for the requested immutable commit and runs approved security tools. Results are normalized, redacted, stored in MongoDB and exposed through dashboard APIs. Where enabled, SecureFlow publishes a summary to the relevant GitHub Check.

MVP focus: dependency vulnerability detection and exposed-secret detection. Infrastructure-as-code scanning is an optional extension only after the core journey is stable.

1.1 Vision and mission

Vision: Make practical, understandable security checks accessible to small development teams by bringing supported findings into one workflow.

Mission: Help teams discover and prioritize selected security risks earlier in the development lifecycle through repository-connected automated scanning.

1.2 Product positioning

SecureFlow is an orchestration, workflow and reporting layer around established scanners. It does not create a new vulnerability-detection engine and does not replace professional application security testing.

1.3 Core value

- Earlier feedback during pull-request development.
- Central visibility across connected repositories and scan history.
- Prioritized findings with location and remediation context.
- GitHub workflow reporting without requiring developers to manually run each scanner.

2. Problem statement and opportunity

Security checks are often spread across command-line tools, CI jobs and separate dashboards. Developers may overlook vulnerable packages or accidentally commit credentials. Teams may lack a consistent way to prioritize findings, understand scan status or review whether issues persist across scans.

Problem	SecureFlow response
Vulnerable dependencies go unnoticed	Run dependency checks against supported manifests and lockfiles.
Credentials are committed by mistake	Run secret detection and redact secret values from output.
Results are fragmented	Normalize supported scanner results into one dashboard.
Prioritization is difficult	Filter and group by severity, repository, category and status.
Checks happen too late	Trigger scans from configured GitHub events and manual requests.
No historical view	Persist scan metadata, summaries and finding observations.

3. Goals, non-goals and success metrics

3.1 MVP goals

- Secure account access and organization/workspace isolation.

- GitHub App installation and repository selection.
- Webhook signature verification and duplicate-event protection.
- Asynchronous scan orchestration using Celery and Redis.
- Dependency scanning and secret scanning in a restricted worker.
- Normalized findings, scan history and dashboard views.
- Basic GitHub Check reporting if time and permissions allow.
- Repeatable evaluation using controlled test repositories.

3.2 Non-goals

- Guaranteeing a repository is vulnerability-free.
- Executing repository scripts, package install hooks, builds or Dockerfiles.
- Autonomous code changes or automatic pull-request fixes.
- Full penetration testing, DAST, runtime protection or complete compliance certification.
- Multi-cloud enterprise orchestration, billing and advanced subscription management.
- GitLab/Bitbucket integration in the first release.

3.3 Success metrics

Metric	Measurement
Scan completion rate	Completed eligible jobs / accepted eligible jobs.
Scan duration	Worker start to completed result; report median and range.
Detection rate	Known seeded test cases detected / total seeded cases.
False-positive review	Manually reviewed findings judged incorrect / reviewed findings.
Queue wait time	Time between enqueue and worker start.
Tenant isolation	All designed cross-workspace access tests are denied.

Set numeric targets after measuring team hardware and representative repositories. Do not claim universal detection percentages before validation.

4. Users, roles and permissions

Role	Responsibilities	Permissions
Organization Owner	Creates workspace, connects GitHub, selects repositories and manages members.	Manage workspace, integration, repositories and membership.
Developer	Reviews repository security and investigates findings.	View authorized repositories, scans and findings; update permitted workflow status.
Platform Admin	Operates SecureFlow infrastructure and platform operations.	Platform operational access; no automatic access to customer source snapshots.

4.1 Authorization principles

- Every workspace-owned record includes organizationId.
- Every protected API verifies identity, workspace membership and role.
- Repository access is limited to repositories granted to the GitHub App installation.
- Organization permissions and platform administration are separate.

- Sensitive integration changes and member changes create audit events.

5. Product scope and prioritization

Priority	Capabilities	Release
Must Have	Authentication, workspace, GitHub App, repository selection, webhook verification, Celery queue, dependency scan, secret scan, findings dashboard, scan history, tenant isolation and cleanup.	MVP
Should Have	GitHub Checks, scan comparison, activity log, finding status workflow and repository analytics.	If core MVP is stable
Future	Container image scanning, broader IaC, notifications, compliance reports, GitLab/Bitbucket, AI explanations/remediation and billing.	Post-MVP

Release rule: prioritize one complete, secure journey—connect repository → trigger scan → process findings → display report → publish status—over a large set of disconnected features.

6. User journeys and system workflow

6.1 First-time setup

- User registers and creates a workspace.
- User installs the SecureFlow GitHub App and authorizes selected repository access.
- SecureFlow associates the installation with the workspace.
- Owner selects repositories and enables monitoring.
- Application shows integration and repository connection status.

6.2 Automated pull-request scan

- Developer opens or updates a pull request in a monitored repository.
- GitHub sends a subscribed webhook event.
- FastAPI webhook route verifies the signature against the raw request body and validates event metadata.
- Delivery ID is checked to prevent duplicate processing.
- Backend creates a scan record and dispatches a Celery task referencing an immutable commit SHA.
- Restricted worker obtains the source snapshot and runs approved scanners.
- Results are normalized, redacted, persisted and summarized.
- Temporary source material is deleted; scan status is finalized.
- Dashboard updates and optional GitHub Check reports the summary.

6.3 Manual scan

An authorized user selects Run Scan on a repository. The API validates membership and repository status, creates a scan record, dispatches a Celery task and immediately returns a scan identifier. The UI displays Queued, Running, Completed, Failed or Timed Out.

6.4 Finding investigation

A developer filters findings, opens a finding detail, reviews severity, category, scanner evidence, file/line location where available and remediation guidance. The developer may update the workflow label to Open, Acknowledged, Resolved or Accepted Risk. Workflow status does not itself prove a vulnerability is fixed.

6.5 End-to-end workflow

Stage	System action	Output
Connect	Install GitHub App; choose repositories.	Authorized repository mapping.
Trigger	Webhook or manual scan request.	Validated scan request.
Queue	Create scan record and dispatch Celery task through Redis.	Queued job and scan ID.
Scan	Restricted worker runs Trivy/Gitleaks.	Scanner output.
Normalize	Validate, redact, map severity and deduplicate.	Normalized findings.
Persist	Store scan summary and finding observations in MongoDB.	History and current state.
Report	Refresh dashboard and optionally publish GitHub Check.	Developer-facing report.

7. Functional requirements

ID	Requirement
FR-01 Authentication	Register, log in/out and retrieve current profile. Hash passwords using a suitable password-hashing algorithm; protect sessions/tokens.
FR-02 Authorization	Check identity, workspace membership and role for every protected operation. Return appropriate 401/403 responses.
FR-03 Workspace	Create and list workspaces; persist owner and membership relationships.
FR-04 GitHub App	Install app and associate installation with workspace. Keep credentials server-side.
FR-05 Repository selection	List installation-accessible repositories; enable/disable monitoring for selected repositories.
FR-06 Webhook intake	Accept subscribed events, verify signature against raw body, validate event type and required fields.
FR-07 Idempotency	Use delivery identifiers and repository/commit context to prevent duplicate scans from repeated webhook deliveries.
FR-08 Scan scheduling	Manual and webhook triggers create scan records and dispatch asynchronous Celery tasks; do not block HTTP request for scan completion.
FR-09 Worker processing	Validate task data, fetch immutable source snapshot, run approved commands and enforce limits.
FR-10 Dependency scan	Use pinned Trivy version/configuration to detect supported dependency vulnerabilities.
FR-11 Secret scan	Use pinned Gitleaks version/configuration; redact secret values from all persisted/displayed/logged output.
FR-12 IaC scan	Optional MVP extension: Trivy configuration checks for explicitly listed and tested formats.
FR-13 Result normalization	Convert scanner outputs to a stable finding schema while preserving source tool, version and rule/advisory identifiers.
FR-14 Deduplication	Use stable fingerprints while preserving scan associations and first/last-seen observations.
FR-15 Dashboard	Show repository totals, scan totals, severity counts, recent activity and current findings.
FR-16 Finding search	Filter and paginate by repository, severity, category and workflow status.
FR-17 Finding detail	Show title, description, severity, location, scanner reference and remediation guidance.
FR-18 Scan history	Show commit SHA, trigger, status, timestamps, duration and summary.
FR-19 GitHub reporting	Publish/update a GitHub Check summary for the scanned commit where configured.

ID	Requirement
FR-20 Audit events	Record important actions such as installation changes, repository monitoring and membership changes.
FR-21 Cleanup	Delete temporary source and scanner workspaces after success, failure, timeout and recovery.
FR-22 Failure handling	Record scanner errors, worker failures and timeouts distinctly; apply bounded retry/backoff using Celery task policies.

8. Frontend and UX requirements

8.1 Information architecture

Section	Screens	Key content
Public	Landing, Features, Docs, Login, Register, Password Recovery	Product overview and account access.
Overview	Dashboard	Summary cards, severity chart, recent scans and repository status.
Repositories	List and detail	Connected repos, monitoring toggle, last scan and findings summary.
Security	Findings list and detail	Search, filters, evidence, location and remediation.
Scans	History and detail	Trigger, commit, status, duration, summary and scanner errors.
Integrations	GitHub connection and repo selection	Installation state, permission context and selected repositories.
Workspace	Members and settings	Workspace details, members and roles.
Account	Profile and security settings	User profile, password/session controls.

8.2 Dashboard components

- Stat cards: monitored repositories, total scans, open findings, critical/high findings.
- Severity distribution chart: Critical, High, Medium, Low and Unknown.
- Recent scans table: repository, commit, trigger, status, time and duration.
- Repository status table: latest scan, open findings and last activity.
- Activity feed for connection, scan and finding workflow events.

8.3 Required states and interactions

- Loading skeletons for cards and tables; clear empty states with Connect GitHub or Run First Scan action.
- Clear error state with retry where safe.
- Queued, Running, Completed, Failed and Timed Out scan states.
- Permission-denied state and confirmation for sensitive actions.
- Responsive desktop/tablet layouts, keyboard navigation and visible focus.
- Pagination, sorting, search and server-side filters.

8.4 Visual design direction

Use a restrained SaaS interface with navy/slate typography, neutral light surfaces, indigo primary actions and teal status accents. Severity colors must also have text labels/icons. Prioritize readable tables and scan status over decorative animation.

8.5 Frontend engineering

- React + TypeScript + Tailwind CSS with reusable typed components.
- Central API client and typed request/response models.
- Route guards for user experience; backend remains authorization authority.
- Form validation, consistent errors, toasts and loading states.
- Never store GitHub private keys, installation tokens or raw secrets in browser storage.

9. Architecture and service boundaries

Layer	Technology	Responsibilities
Frontend	React, TypeScript, Tailwind	Authentication UI, workspace, repositories, dashboard, findings and scan history.
Backend API	Python, FastAPI, Pydantic	Authentication, authorization, workspace, GitHub App, webhook validation, scan orchestration and REST APIs.
Queue/broker	Celery + Redis	Background task dispatch, broker transport, retries, task routing and queue operations.
Scanner worker	Python Celery worker	Fetch source snapshot, invoke pinned tools, parse output, enforce limits and cleanup.
Security tools	Trivy, Gitleaks	Dependency vulnerability and secret detection; optional IaC checks.
Database	MongoDB + PyMongo	Users, workspaces, memberships, installations, repositories, scans, findings and audit events.
Runtime/CI	Docker, Docker Compose, GitHub Actions	Repeatable environments, tests, image builds and deployment.

9.1 Logical architecture

Developer → GitHub → GitHub App webhook → FastAPI API → Celery task dispatch → Redis broker → restricted Python Celery worker → Trivy/Gitleaks → result normalizer → MongoDB → dashboard and optional GitHub Check.

9.2 Service boundaries

- Frontend talks only to backend APIs; it never accesses MongoDB or Redis directly.
- FastAPI validates and dispatches tasks; it does not execute repository content.
- Celery worker processes scans in a separate restricted runtime/container.
- Scanner output is untrusted input and must be validated before storage/rendering.
- GitHub reporting uses narrowly scoped installation credentials.

9.3 Conceptual Celery task payload

Include scanId, organizationId, repositoryId, installation reference, repository identifier, immutable commit SHA, trigger type and correlation/delivery ID. Do not put private keys, long-lived tokens or raw secrets in task arguments.

9.4 Scan state machine

State	Meaning	Next state
Queued	Accepted and waiting for worker.	Running, Cancelled, Failed
Running	Worker has claimed task.	Completed, Failed, Timed Out
Completed	Scanners finished and results persisted.	Terminal

State	Meaning	Next state
Failed	Unexpected processing/scanner error.	Bounded retry or terminal
Timed Out	Configured time limit exceeded.	Bounded retry or terminal
Cancelled	Cancelled before execution where supported.	Terminal

10. Scanning engine and finding lifecycle

Scanner	Purpose	Scope and limitations
Trivy	Dependency vulnerability scanning against supported manifests/lockfiles.	Pin version and configuration; document supported file types and severity mapping.
Gitleaks	Detect likely credentials and secrets in repository content/history according to configured mode.	Pin version; redact secret values; carefully manage allowlists; use dummy test secrets.
Trivy config	Detect supported IaC/configuration misconfigurations.	Optional extension; state supported formats and tested checks explicitly.

10.1 Safe scanner execution

- Treat every repository as untrusted input.
- Use temporary workspace and non-root scanner container.
- Apply CPU, memory, process, file-size and wall-clock limits.
- Disable network access during scanning where practical.
- Do not execute package lifecycle scripts, project scripts, builds, Dockerfiles or arbitrary binaries.
- Never mount host Docker socket or sensitive host directories.
- Use short-lived least-privilege credentials to fetch private source; expire/revoke promptly.
- Clean temporary files on success, failure, timeout and worker termination.

10.2 Normalized finding fields

Field	Description
findingId; organizationId; repositoryId; scanId	Internal identity and tenant/repository/scan relationship.
fingerprint	Stable identity for deduplication.
title; description; severity; category	Human-readable details; Critical/High/Medium/Low/Unknown; Dependency/Secret/IaC.
sourceTool; toolVersion; ruleId	Scanner provenance and rule/advisory identifier.
packageName; installedVersion; fixedVersion	Dependency-specific details when applicable.
filePath; lineNumber	Location evidence when supplied by scanner.
redactedEvidence	Safe contextual evidence; never raw credential value.
remediation; references	Scanner-provided or curated next steps and reference links.
status; firstSeenAt; lastSeenAt	Workflow state and observation history.

10.3 Severity and lifecycle rules

Preserve original scanner severity and source metadata. Map to the platform's five display levels without silently treating Unknown as Low. A fingerprint should combine stable identity inputs such as repository, scanner/rule, package or secret

rule and suitable file context. A finding may appear in multiple scans. Do not mark it Resolved merely because it is absent from a partial or failed scan.

11. Data model and retention

Collection	Key fields	Rules
users	name, email, passwordHash/identity reference, createdAt	Unique normalized email; no plaintext passwords.
organizations	name, ownerId, plan, createdAt	Workspace root.
memberships	organizationId, userId, role, joinedAt	Unique membership per workspace/user.
githubInstallations	organizationId, installationId, encrypted credential reference	No plaintext long-lived tokens/private keys.
repositories	organizationId, GitHub repository ID, owner/name, defaultBranch, enabled, lastScannedAt	Index repository identity and tenant.
scans	organizationId, repositoryId, commitSha, triggerType, status, timestamps, duration, summary	Index by tenant, repository, status and date.
findings	organizationId, repositoryId, scanId, fingerprint, severity, category, filePath, status	Index for filters and fingerprint lookup.
auditEvents	organizationId, actorId, action, resourceType, resourceId, timestamp	Append-oriented record for important actions.

11.1 Tenant isolation

All organization-owned records include organizationId. Every query includes the authorized tenant scope. Never trust a workspace ID supplied by the browser without verifying membership. Add automated negative tests for cross-workspace reads and writes.

11.2 Retention

- Temporary repository snapshots: delete immediately after scan completion or failure cleanup.
- Raw scanner output: retain only if needed and after redaction; otherwise retain normalized results only.
- Scan/finding metadata and audit events: define and document project retention durations.
- GitHub credentials: encrypted or held in a secrets manager; support disconnect, revocation and deletion.

12. API requirements

Proposed base path: /api/v1. Use JSON, Pydantic request/response schemas, server-side validation, pagination and consistent error envelopes. FastAPI's generated OpenAPI schema should be maintained as the API contract.

Method	Endpoint	Purpose
POST	/auth/register	Create account.
POST	/auth/login	Authenticate.
POST	/auth/logout	End session.
POST	/auth/refresh	Refresh session if used.
GET	/auth/me	Current user.
POST	/organizations	Create workspace.
GET	/organizations	List workspaces.

Method	Endpoint	Purpose
GET	/organizations/{id}	Workspace detail.
GET	/organizations/{id}/members	List members.
GET	/github/installations	List linked installations.
GET	/github/repositories	List accessible repositories.
POST	/github/repositories/connect	Enable selected repository.
DELETE	/github/repositories/{id}	Disable monitoring.
POST	/webhooks/github	Receive GitHub event.
GET	/repositories	List monitored repositories.
GET	/repositories/{id}	Repository detail.
GET	/repositories/{id}/scans	Repository scan history.
POST	/scans	Start manual scan.
GET	/scans	List scans.
GET	/scans/{id}	Scan status/summary.
GET	/scans/{id}/findings	Findings for scan.
GET	/findings	Search/filter findings.
GET	/findings/{id}	Finding detail.
PATCH	/findings/{id}/status	Update workflow status.
GET	/dashboard/overview	Workspace metrics.
GET	/dashboard/activity	Recent activity.

12.1 API conventions

- Use pagination with a documented maximum page size.
- Support filters such as repositoryId, severity, category, status, date range and sort.
- Return 401 for unauthenticated and 403 for authenticated-but-unauthorized requests.
- Use request/correlation IDs for tracing.
- Use idempotency for scan creation and webhook processing.
- Do not expose internal stack traces or secrets in error responses.
- Publish OpenAPI documentation and example payloads.

12.2 Example response


```
{"scanId":"scan_123","status":"queued","repositoryId":"repo_456","commitSha":"abc123","createdAt":"2026
-11-05T10:00:00Z"}
```


13. Security, privacy and threat controls

Threat	Required control
Forged webhook	Verify HMAC signature against raw body; reject invalid signatures; validate event type.
Replay/duplicate delivery	Record delivery identifier and enforce idempotency.

Threat	Required control
Token/private key exposure	Keep credentials server-side, encrypt at rest/use secrets manager, restrict access and rotate/revoke.
Malicious repository content	Never execute project code; isolate scanner; enforce resource limits and cleanup.
Container/host exposure	Non-root, minimal capabilities, no privileged mode, no host Docker socket, restricted mounts.
Cross-tenant access	Tenant scope every query and authorize every protected operation.
Secret leakage	Redact values before logging, storing display evidence or rendering UI.
Resource exhaustion	Rate limit; cap repository size, file count, duration, concurrency and retries.
Supply-chain risk	Pin tool/base-image versions and update through reviewed process.
XSS/injection	Escape untrusted scanner text, validate input and avoid unsafe HTML rendering.

13.1 Minimum security baseline

- TLS for deployed traffic; secure session/cookie or short-lived token configuration.
- CSRF protection if cookie-based authentication is used; strict CORS allowlist.
- Rate limits, request-size limits, security headers and safe errors.
- Audit events for integration and permission changes.
- Separate test and production credentials.
- Backups and a tested restore procedure for persistent metadata.

13.2 Privacy

Repository contents and credentials are sensitive. Explain what is fetched, why it is processed, whether any external service is contacted, what metadata is retained and when temporary files are deleted. Do not send repository contents to external services unless the product explicitly discloses and secures that behavior.

14. Non-functional requirements

Area	Requirement
Security	Authorized workspace access only; protected credentials; isolated scanner execution.
Performance	Long scans run asynchronously; dashboard lists are paginated.
Reliability	Explicit task states, bounded retries, timeout handling and failure records.
Scalability	API and Celery workers scale independently; worker concurrency configurable.
Maintainability	Modular Python packages, typed interfaces, documented APIs and automated tests.
Usability	Findings explain severity, location and next steps in understandable language.
Privacy	Temporary source snapshots are removed under a defined retention policy.
Observability	Structured logs, request IDs, queue metrics, worker health and scan metrics.

15. Testing and acceptance criteria

Test area	Cases	Expected result
Authentication	Valid/invalid login, logout, expired session, protected API.	Correct access behavior; no credential leakage.

Test area	Cases	Expected result
Workspace	Create workspace, membership, role restrictions.	Only authorized members can access data.
GitHub	Install app, select repo, revoke installation.	Integration state remains accurate.
Webhook	Valid/invalid signature, duplicate, unsupported event.	Only valid supported events processed once.
Celery/Redis	Worker restart, retry, timeout, broker unavailable.	Safe recovery or explicit failure state.
Dependency scan	Known vulnerable dependency fixtures.	Expected supported findings detected and stored.
Secret scan	Controlled dummy credentials in test files.	Seeded secrets detected and redacted.
IaC optional	Known misconfiguration fixtures in supported formats.	Only documented checks are evaluated.
Findings UI	Filters, pagination, details, status update.	Correct results and authorization.
Tenant isolation	Two workspaces; attempt cross-tenant reads/writes.	All unauthorized attempts denied.
Cleanup	Success, failure, timeout and worker termination.	Temporary source data removed or recovered by cleanup.
Performance	Small, medium and larger test repositories.	Duration, queue wait and resource use measured.

15.1 MVP acceptance checklist

- User registers, logs in and creates a workspace.
- Owner installs GitHub App and selects a test repository.
- Webhook signature verification and duplicate-delivery handling work.
- Manual scan and supported GitHub event dispatch Celery tasks.
- Worker scans immutable source without executing repository code.
- Trivy and Gitleaks produce normalized findings on controlled fixtures.
- Secret values are redacted from logs, UI and display-oriented storage.
- Dashboard, filters and scan history display correct data.
- Cross-tenant authorization tests pass.
- Temporary snapshots are deleted after success and failure.
- GitHub Check summary works if included in final MVP scope.
- Deployment guide, test evidence and technical documentation are complete.

16. Deployment and operations

16.1 Development environment

- Recommended: 8-core CPU, 16 GB RAM and 20–30 GB free SSD space.
- Minimum workable: 4-core CPU, 8 GB RAM and 10–15 GB free storage; Docker may be slower.
- Tools: VS Code, Git, Python 3, Node.js LTS, MongoDB, Redis and Docker.

16.2 Local deployment

Use Docker Compose for frontend, FastAPI API, MongoDB, Redis and the separate Celery scanner worker. Run scanner execution in a restricted container/runtime profile. Keep development secrets in environment files excluded from version control.

16.3 CI/CD

- On pull request: install dependencies, lint, type-check where configured, run tests and build frontend.
- Run FastAPI and scanner-worker tests.
- Build container images and scan images/dependencies where practical.
- Deploy only from protected branches or approved release workflow.
- Store CI credentials in secret storage, never in repository files.

16.4 Operations visibility

- Health endpoints for API and worker heartbeat.
- Structured logs with correlation IDs; never log tokens or raw secret findings.
- Queue depth, failed tasks, worker heartbeat and scan duration metrics.
- Track scanner exit codes, timeout count and finding totals.
- Provide an operational warning for repeated failures or queue backlog.

17. Six-month implementation roadmap

Month	Workstream	Deliverable
1 — Plan/design	Finalize scope, architecture, data model, threat model, wireframes, repo structure and environment.	Approved PRD, architecture, wireframes and API draft.
2 — Foundation	Frontend shell/design system, FastAPI foundation, authentication, workspace, MongoDB models and protected APIs.	Working application foundation.
3 — GitHub	GitHub App, installation flow, repository selection, webhook validation and idempotency.	Connected test repository and verified webhook.
4 — Scanning	Celery/Redis, Python worker, source retrieval, Trivy, Gitleaks and result normalization.	Background scan with persisted findings.
5 — Dashboard	Findings, filters, history, dashboard metrics, GitHub Checks and end-to-end integration.	Demonstrable MVP.
6 — Harden/deliver	Security/performance tests, tenant isolation, cleanup, deployment and documentation.	Deployed MVP and final project package.

17.1 Recommended implementation phases

Phase	Work	Exit criterion
0	Repository, environment, architecture, Docker Compose.	Services start locally.
1	FastAPI foundation, MongoDB connection, health checks, Pydantic settings.	API starts and database health is verified.
2	Authentication, workspaces, membership and role authorization.	Protected workspace APIs pass tests.
3	GitHub App installation and repository selection.	Test repository is connected.
4	Webhook validation, idempotency and scan record creation.	Valid event creates one scan.
5	Celery/Redis task dispatch and task lifecycle.	Manual scan becomes a queued task and status updates.
6	Restricted worker, source snapshot, Trivy and Gitleaks.	Controlled fixture yields expected findings.
7	Finding normalization, deduplication, redaction and persistence.	Normalized findings are stored safely.
8	Dashboard APIs and frontend integration.	Real findings and scan history appear in UI.

Phase	Work	Exit criterion
9	GitHub Checks, security hardening, testing and deployment.	End-to-end MVP demo succeeds.

17.2 Suggested team split

Member	Ownership	Responsibilities
Student 1	Frontend	React screens, design system, dashboard, findings, API integration and UI tests.
Student 2	Python backend	FastAPI APIs, auth, workspace, MongoDB, GitHub App and webhook.
Student 3	Scanning	Python worker, Trivy/Gitleaks, result normalization, isolation and test fixtures.
Student 4 (if available)	DevOps/integration	Docker, Redis/Celery, CI/CD, GitHub Checks, deployment and integration tests.

All team members should understand the end-to-end architecture and be able to explain the code-change-to-security-report journey.

18. Risks, assumptions and dependencies

Risk / assumption	Impact	Mitigation
GitHub App permissions/setup take time	Integration delays.	Create test app early; request minimum permissions and validate events.
Private repository access mishandled	Source exposure.	Use installation-scoped short-lived credentials and strict cleanup.
Scanner output varies by version/config	Unstable results.	Pin versions, store provenance and test fixtures.
False positives or missed issues	Reduced trust.	Document coverage and evaluate controlled fixtures; do not promise perfect detection.
Worker resource exhaustion	Queue delays or service instability.	Apply limits, concurrency controls, timeout and bounded retry.
Scope grows beyond six months	Incomplete MVP.	Freeze Must Have scope; defer AI, multi-provider and advanced compliance.
Team skill gaps	Implementation delays.	Build a vertical slice early and document service contracts.

18.1 Assumptions

- GitHub is the initial source-control provider.
- The team can create and configure a GitHub App for testing.
- MVP scanning runs on controlled infrastructure with Docker available.
- MongoDB, Redis, Python and scanner tools are available in the development/deployment environment.
- The initial target is a demonstrable academic MVP, not a production-certified enterprise security service.

18.2 Dependencies

- GitHub App registration, permissions and webhook configuration.
- Trivy and Gitleaks version/configuration selection.
- Redis availability and Celery worker lifecycle.
- Secure storage for GitHub credentials and application secrets.

• Representative test repositories and controlled vulnerability/secret fixtures.

19. Deliverables and final demonstration

19.1 Project deliverables

• Complete source code with frontend, FastAPI backend and scanner worker.

• Architecture diagram, data model and API/OpenAPI documentation.

• GitHub App setup and configuration guide.

• Docker Compose development environment and deployment instructions.

• Automated tests and controlled scanner test fixtures.

• Security and performance evaluation report.

• User guide, known limitations and retention/privacy description.

• Final presentation and recorded/live demonstration.

19.2 Suggested final demo

• Register and create a workspace.

• Connect a test GitHub repository.

• Open/update a pull request containing a controlled vulnerable dependency and dummy secret.

• Show webhook receipt and Celery task creation.

• Show worker execution and scanner completion.

• Open SecureFlow dashboard and inspect severity summary.

• Open finding detail and demonstrate redaction/remediation context.

• Show scan history and GitHub Check result.

• Demonstrate tenant isolation or invalid webhook rejection.

Use only test repositories and fake credentials in the demonstration. Never place real API keys or passwords in screenshots, reports or demo commits.

20. Glossary and technical references

Term	Meaning
DevSecOps	Integrating security practices and automated checks into software development and delivery.
SAST	Static Application Security Testing; analysis of source code for certain classes of weaknesses.
SCA	Software Composition Analysis; assessment of third-party dependencies and known vulnerabilities.
Secret scanning	Detection of patterns that may represent credentials or tokens.
Webhook	An HTTP event notification sent by an external platform such as GitHub.
GitHub App	An integration identity with configurable repository permissions and installation-scoped access.
FastAPI	Python web framework used to implement SecureFlow's backend REST API.
Celery	Python distributed task queue used to dispatch and execute asynchronous scan tasks.
Redis	In-memory data store used as the Celery broker in the MVP architecture.
Finding fingerprint	A stable identifier used to recognize the same issue across scans.
Tenant isolation	Ensuring one organization cannot access another organization's data.

Official technical references

- **FastAPI documentation:** https://fastapi.tiangolo.com/
- **Pydantic documentation:** https://docs.pydantic.dev/
- **Celery documentation:** https://docs.celeryq.dev/
- **Redis documentation:** https://redis.io/docs/
- **PyMongo documentation:** https://pymongo.readthedocs.io/
- **GitHub Apps documentation:** https://docs.github.com/en/apps
- **GitHub webhook security:** https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries
- **GitHub Checks API:** https://docs.github.com/en/rest/checks
- **Trivy documentation:** https://trivy.dev/latest/docs/
- **Gitleaks repository:** https://github.com/gitleaks/gitleaks
- **Docker security documentation:** https://docs.docker.com/engine/security/
- **OWASP ASVS:** https://owasp.org/www-project-application-security-verification-standard/
- **OWASP Secrets Management Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Final product definition: SecureFlow is a GitHub-connected DevSecOps platform that uses a Python FastAPI backend to orchestrate selected security scanners asynchronously through Celery and Redis, processes findings in restricted workers, and provides developers with a centralized view of repository security risks before code is merged.